

Event Management App

JSP + Servlet + JDBC, following the MVC pattern

Assumptions: MySQL database, Tomcat 9 (javax.servlet) and JSTL for the list page. For Tomcat 10+, change javax.servlet to jakarta.servlet. Each file has a comment at the top stating its role in MVC (Question 5).

Project Structure

```
src/main/java/  
  controller/ RegisterUserServlet, LoginServlet, LogoutServlet, EventPostServlet, EventListServlet  
  model/      User, Event, UserDAO, EventDAO  
  util/       DBUtil  
src/main/webapp/ (views)  
  RegisterUser.jsp, Login.jsp, Home.jsp, EventPost.jsp, EventsList.jsp
```

Database Setup

```
CREATE DATABASE eventdb;  
USE eventdb;  
  
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  password VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE events (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(100) NOT NULL,  
  event_date DATE NOT NULL,  
  description TEXT,  
  posted_by VARCHAR(50)  
);
```

Utility: Database Connection

util/DBUtil.java

```
// util/DBUtil.java  
// Helper class: provides JDBC connections to the model layer.  
package util;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
  
public class DBUtil {  
  private static final String URL = "jdbc:mysql://localhost:3306/eventdb";  
  private static final String USER = "root";  
  private static final String PASS = "your_password";  
  
  public static Connection getConnection() throws SQLException, ClassNotFoundException {  
    Class.forName("com.mysql.cj.jdbc.Driver");  
    return DriverManager.getConnection(URL, USER, PASS);  
  }  
}
```

Models (JavaBeans and DAOs)

model/User.java

```
// model/User.java  
// MODEL: JavaBean representing a user.  
package model;
```

```

public class User {
    private int id;
    private String username, email, password;

    public User() {}
    public User(String username, String email, String password) {
        this.username = username; this.email = email; this.password = password;
    }
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}

```

model/Event.java

```

// model/Event.java
// MODEL: JavaBean representing one event (required by Q4).
package model;

import java.sql.Date;

public class Event {
    private int id;
    private String title, description, postedBy;
    private Date eventDate;

    public Event() {}
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public Date getEventDate() { return eventDate; }
    public void setEventDate(Date eventDate) { this.eventDate = eventDate; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public String getPostedBy() { return postedBy; }
    public void setPostedBy(String postedBy) { this.postedBy = postedBy; }
}

```

model/UserDAO.java

```

// model/UserDAO.java
// MODEL: handles all database operations for the users table.
package model;

import java.sql.*;
import util.DBUtil;

public class UserDAO {

    // Insert a new user (used by RegisterUserServlet)
    public boolean register(User u) throws Exception {
        String sql = "INSERT INTO users (username, email, password) VALUES (?, ?, ?)";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, u.getUsername());
            ps.setString(2, u.getEmail());
            ps.setString(3, u.getPassword());
            return ps.executeUpdate() > 0;
        }
    }

    // Check username + password (used by LoginServlet). Returns null if invalid.
    public User authenticate(String username, String password) throws Exception {
        String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, username);
            ps.setString(2, password);
            try (ResultSet rs = ps.executeQuery()) {

```

```

        if (rs.next()) {
            User u = new User();
            u.setId(rs.getInt("id"));
            u.setUsername(rs.getString("username"));
            u.setEmail(rs.getString("email"));
            return u;
        }
    }
}
return null;
}
}

```

model/EventDAO.java

```

// model/EventDAO.java
// MODEL: handles all database operations for the events table.
package model;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import util.DBUtil;

public class EventDAO {

    public boolean addEvent(Event e) throws Exception {
        String sql = "INSERT INTO events (title, event_date, description, posted_by) VALUES (?, ?, ?, ?)";
        try (Connection con = DBUtil.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, e.getTitle());
            ps.setDate(2, e.getEventDate());
            ps.setString(3, e.getDescription());
            ps.setString(4, e.getPostedBy());
            return ps.executeUpdate() > 0;
        }
    }

    public List<Event> getAllEvents() throws Exception {
        List<Event> list = new ArrayList<>();
        String sql = "SELECT * FROM events ORDER BY event_date";
        try (Connection con = DBUtil.getConnection();
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                Event e = new Event();
                e.setId(rs.getInt("id"));
                e.setTitle(rs.getString("title"));
                e.setEventDate(rs.getDate("event_date"));
                e.setDescription(rs.getString("description"));
                e.setPostedBy(rs.getString("posted_by"));
                list.add(e);
            }
        }
        return list;
    }
}

```

Question 1: Registration

RegisterUser.jsp

```

<!-- RegisterUser.jsp -->
<!-- VIEW: sign-up form. Posts data to the RegisterUserServlet (controller). -->
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<head><title>Register</title></head>
<body>
    <h2>Sign Up</h2>
    <% if (request.getParameter("error") != null) { %>
        <p style="color:red;">Registration failed. Username may already exist.</p>
    <% } %>
    <form action="RegisterUserServlet" method="post">
        Username: <input type="text" name="username" required><br><br>

```

```

        Email:    <input type="email" name="email" required><br><br>
        Password: <input type="password" name="password" required><br><br>
        <input type="submit" value="Register">
    </form>
    <p>Already have an account? <a href="Login.jsp">Login</a></p>
</body>
</html>

```

controller/RegisterUserServlet.java

```

// controller/RegisterUserServlet.java
// CONTROLLER: receives the registration form, builds a User model, saves it via UserDAO.
package controller;

import java.io.IOException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import model.User;
import model.UserDAO;

@WebServlet("/RegisterUserServlet")
public class RegisterUserServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        User u = new User(req.getParameter("username"),
                           req.getParameter("email"),
                           req.getParameter("password"));

        try {
            if (new UserDAO().register(u)) {
                resp.sendRedirect("Login.jsp?registered=1");
            } else {
                resp.sendRedirect("RegisterUser.jsp?error=1");
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            resp.sendRedirect("RegisterUser.jsp?error=1");
        }
    }
}

```

Question 2: Login (JDBC + HttpSession)

Login.jsp

```

<!-- Login.jsp --%>
<!-- VIEW: login form. Posts to LoginServlet (controller). --%>
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<head><title>Login</title></head>
<body>
    <h2>Login</h2>
    <% if (request.getParameter("registered") != null) { %>
        <p style="color:green;">Registration successful. Please log in.</p>
    <% } %>
    <% if (request.getParameter("error") != null) { %>
        <p style="color:red;">Invalid username or password.</p>
    <% } %>
    <form action="LoginServlet" method="post">
        Username: <input type="text" name="username" required><br><br>
        Password: <input type="password" name="password" required><br><br>
        <input type="submit" value="Login">
    </form>
    <p>New user? <a href="RegisterUser.jsp">Register here</a></p>
</body>
</html>

```

controller/LoginServlet.java

```

// controller/LoginServlet.java
// CONTROLLER: authenticates the user via UserDAO and stores login state in HttpSession.
package controller;

import java.io.IOException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

```

```

import model.User;
import model.UserDAO;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        String username = req.getParameter("username");
        String password = req.getParameter("password");
        try {
            User user = new UserDAO().authenticate(username, password);
            if (user != null) {
                HttpSession session = req.getSession(); // create session
                session.setAttribute("user", user); // store login state
                resp.sendRedirect("Home.jsp");
            } else {
                resp.sendRedirect("Login.jsp?error=1");
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            resp.sendRedirect("Login.jsp?error=1");
        }
    }
}

```

controller/LogoutServlet.java

```

// controller/LogoutServlet.java
// CONTROLLER: ends the session (logout).
package controller;

import java.io.IOException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) session.invalidate();
        resp.sendRedirect("Login.jsp");
    }
}

```

Home.jsp

```

<!-- Home.jsp --%>
<!-- VIEW: landing page shown after successful login. Protected by session check. --%>
<%@ page contentType="text/html; charset=UTF-8" import="model.User" %>
<%
    User user = (User) session.getAttribute("user");
    if (user == null) { response.sendRedirect("Login.jsp"); return; }
%>
<html>
<head><title>Home</title></head>
<body>
    <h2>Welcome, <%= user.getUsername() %>!</h2>
    <ul>
        <li><a href="EventPost.jsp">Post a new event</a></li>
        <li><a href="EventListServlet">View all events</a></li>
        <li><a href="LogoutServlet">Logout</a></li>
    </ul>
</body>
</html>

```

Question 3: Post an Event

EventPost.jsp

```

<!-- EventPost.jsp --%>
<!-- VIEW: form to create an event. Only accessible to logged-in users. --%>
<%@ page contentType="text/html; charset=UTF-8" import="model.User" %>
<%
    if (session.getAttribute("user") == null) { response.sendRedirect("Login.jsp"); return; }
%>

```

```

<html>
<head><title>Post Event</title></head>
<body>
  <h2>Post a New Event</h2>
  <% if (request.getParameter("error") != null) { %>
    <p style="color:red;">Could not save the event. Try again.</p>
  <% } %>
  <form action="EventPostServlet" method="post">
    Title: <input type="text" name="title" required><br><br>
    Date: <input type="date" name="date" required><br><br>
    Description:<br>
    <textarea name="description" rows="5" cols="40"></textarea><br><br>
    <input type="submit" value="Post Event">
  </form>
  <p><a href="Home.jsp">Back to Home</a></p>
</body>
</html>

```

controller/EventPostServlet.java

```

// controller/EventPostServlet.java
// CONTROLLER: checks login, builds an Event model from the form, saves it via EventDAO.
package controller;

import java.io.IOException;
import java.sql.Date;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import model.Event;
import model.EventDAO;
import model.User;

@WebServlet("/EventPostServlet")
public class EventPostServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {
        HttpSession session = req.getSession(false);
        User user = (session == null) ? null : (User) session.getAttribute("user");
        if (user == null) { resp.sendRedirect("Login.jsp"); return; }

        try {
            Event e = new Event();
            e.setTitle(req.getParameter("title"));
            e.setEventDate(Date.valueOf(req.getParameter("date"))); // format yyyy-MM-dd
            e.setDescription(req.getParameter("description"));
            e.setPostedBy(user.getUsername());

            if (new EventDAO().addEvent(e)) {
                resp.sendRedirect("EventListServlet");
            } else {
                resp.sendRedirect("EventPost.jsp?error=1");
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            resp.sendRedirect("EventPost.jsp?error=1");
        }
    }
}

```

Question 4: List All Events (JavaBean + Request Attributes)

controller/EventListServlet.java

```

// controller/EventListServlet.java
// CONTROLLER: fetches events from the model (EventDAO) and forwards them to the view
// as a request attribute.
package controller;

import java.io.IOException;
import java.util.List;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import model.Event;
import model.EventDAO;

```

```

@WebServlet("/EventListServlet")
public class EventListServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        try {
            List<Event> events = new EventDAO().getAllEvents();
            req.setAttribute("eventList", events); // pass data to JSP
            req.getRequestDispatcher("EventsList.jsp").forward(req, resp);
        } catch (Exception ex) {
            throw new ServletException(ex);
        }
    }
}

```

EventsList.jsp

```

<!-- EventsList.jsp -->
<!-- VIEW: displays the list of Event beans received from EventListServlet. -->
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<head><title>All Events</title></head>
<body>
<h2>All Events</h2>
<table border="1" cellpadding="6">
<tr>
<th>Title</th><th>Date</th><th>Description</th><th>Posted By</th>
</tr>
<c:forEach var="e" items="${eventList}">
<tr>
<td>${e.title}</td>
<td>${e.eventDate}</td>
<td>${e.description}</td>
<td>${e.postedBy}</td>
</tr>
</c:forEach>
<c:if test="${empty eventList}">
<tr><td colspan="4">No events found.</td></tr>
</c:if>
</table>
<p><a href="Home.jsp">Back to Home</a></p>
</body>
</html>

```

Question 5: How the MVC Pattern Is Applied

Layer	Files	Role
Model	User, Event (JavaBeans), UserDao, EventDAO	Hold data and run all JDBC/database logic
View	RegisterUser.jsp, Login.jsp, Home.jsp, EventPost.jsp, EventsList.jsp	Display forms and data only, with no database code
Controller	RegisterUserServlet, LoginServlet, LogoutServlet, EventPostServlet, EventListServlet	Handle requests, call the model, and choose which view to show

Request flow: Browser → Servlet (Controller) → DAO/Bean (Model) → request attribute → JSP (View) → Browser.

Required Libraries

Put these in WEB-INF/lib (or Maven dependencies): mysql-connector-j and jstl (1.2).

Note

For simplicity, passwords are stored as plain text. A real application should hash passwords (for example with BCrypt) before saving them.